基础模块02_人机接口：任务分工与交付说明

项目：项目班1-1 / 直接测周法频率计

项目负责人及提交人：蒋观海

阶段任务负责人：何玉

版本 / 日期：V1.1 / 2026-10-09

交付状态：驱动已编译并完成主机逻辑测试；原理图待整改及EDA DRC；尚未上板验证。

1. 本阶段分工

沿用现有项目启动分工中显示按键负责人何玉、底层驱动负责人吕朋聪的安排。本阶段按课程说明命名为“基础模块02_人机接口”，替代旧分工稿中的“基础03 显示按键与扩展接口”。以下为责任分工，成员应审阅确认，不能作为已召开评审会或已完成实测的证明。

成员	本阶段负责内容	对应图纸 / 代码
何玉	阶段统筹；独立按键与OLED接口原理图；显示页面设计；组织评审和交付	按键和OLED模块绘制者均署何玉；hmi_demo.c中的render_page
吕朋聪	按键扫描与消抖；OLED驱动；GPIO和硬件I2C1硬件适配	key.h/key.c、oled.h/oled.c、hmi_port.h/hmi_port.c
蒋观海	引脚和架构复核；节拍与演示工程集成；技术终审及提交	hmi_demo.c中的main、SysTick_Handler；HMI.uvprojx
张桂河	核对3.3V供电、I2C上拉、实际OLED针序、按键电气状态	电气评审意见和后续上板电压 / 波形记录
蓬晶晶	独立核对图纸规范、驱动注释和文件顺序；执行测试与归档	检查表、问题关闭记录、测试日志、提交包

函数按责任成员署名。代码为辅助生成实现，由对应责任成员复核后提交；现有原理图仍保留原导出图的署名，不擅自更改实际绘制者信息。

2. 原理图与接口基线

基线文件：SCH_Schematic1_1_2026-10-09.pdf，3页，使用第1页主控图的人机接口连接。完整原始EDA导出PDF放在合并材料末尾，保持原图不变。

功能	器件 / MCU端口	LQFP48脚号	原图网络名	电气 / 软件约定
时基档位键	SW1 / PA5	15	SW1	低有效；固件内部上拉；20ms消抖
模式键	SW2 / PA6	16	SW2	低有效；固件内部上拉；20ms消抖
运行 / 保持键	SW3 / PA7	17	SW3	低有效；固件内部上拉；20ms消抖
硬件复位键	SW4 / NRST	7	SW4	接复位链；不作为第4个可扫描功能键
OLED时钟	OLED1 / PB10	21	OLED_SCL	硬件I2C1复用开漏；R1=4.7kΩ上拉至+3V3
OLED数据	OLED1 / PB11	22	OLED_SDA	硬件I2C1复用开漏；R2=4.7kΩ上拉至+3V3
OLED电源	OLED1	不适用	+3V3 / GND	原图针序GND、VCC、SCL、SDA；需实物复核

用户已确认屏幕为SSD1306、128×64、7位I2C地址0x3C。初始化采用模块内置电荷泵及页寻址。参考Solomon Systech SSD1306数据手册第8.1.5节和第9、10节：https://cdn.sparkfun.com/assets/d/a/a/b/c/SSD1306_datasheet.pdf 。控制器IC的供电脚定义不能代替成品模块针序和额定电压。

3. 原理图尚需整改

1. 建立独立“基础模块02_人机接口”层次页，与主控等模块通过明确端口连接；现图人机接口仍混在主控页。

2. 按键、OLED分别加虚线框，注明功能、低有效 / 3.3V I2C接口及实际负责成员。若何玉负责重绘，应在完成重绘后署名何玉。

3. 标题栏填写完整项目名、图名、实际设计者、日期、版本；目前绘制栏未体现本任务成员，不能满足分工一致性要求。

4. 推荐把SW1/SW2/SW3改为KEY_TIMEBASE/KEY_MODE/KEY_HOLD，把复位网络SW4改为NRST。跨页必须同步修改；现代码按GPIO位置读取，不依赖文字网络名。

5. 图纸注明三键使用MCU内部上拉；按键四脚内部常闭脚对需按实际器件核对。原图的连线不能代替器件封装和实物核对。

6. 标注OLED完整型号、分辨率、地址及实际接口针序，核对是否自带上拉；确认电源入口去耦和电流预算。

7. 团队名称按“桂林电子科技大学（教师姓名）”落实。现无教师姓名和团队工程访问资料，未执行团队设置或工程交付。

8. 在原始立创EDA工程运行DRC，处理悬空脚、连接、跨页网络问题；负责人组织成员评审并记录修改结果，再交付团队工程、重新导出PDF替换附件。

PDF视觉检查仅能确认上述可见信息，不能确认工程网络拓扑、DRC状态、器件封装、实际团队模式或评审完成。

4. 驱动结构与使用

应用层hmi_demo.c → key/oled统一接口 → hmi_port硬件适配 → GD32标准外设库。应用层不直接读写GPIO。每个代码文件首行说明模块和功能；所有函数含功能、参数、返回、注意事项、作者 / 责任成员、版本 / 日期和修改历史。

- Key_Init：初始化三键及状态；Key_Scan(now_ms)：每1~5ms扫描，稳定20ms确认按下 / 释放；Key_GetValue：取KEY_TIMEBASE、KEY_MODE、KEY_HOLD或KEY_NONE。

- 长按仅产生一次按下事件，释放稳定20ms后才允许下一次触发。允许同时按键，按键号依次返回。每键只有一位待处理事件，应用未取走时，同键多次按下合并。

- OLED_Init、OLED_Clear、OLED_Refresh、OLED_ShowChar、OLED_ShowString、OLED_ShowInt、OLED_ShowFloat构成统一显示接口。Clear和Show系列只修改缓冲；必须Refresh才实际更新屏幕。

- 字符5×7，含间距宽6像素，8页×最多21字符；完整可打印ASCII 32~126，其他字节显示问号；不提供汉字。page=0~7，x为像素坐标，字符串不自动换行，超界返回错误且不写入。

- 整数支持int32全范围；浮点固定小数位0~6、半值远离零舍入，拒绝NaN / 无穷及放大后超过约42.95亿的值；小数位越多，可显示数值范围越小。覆盖旧长数字前清屏或补空格。

- 硬件I2C1使用PB10/PB11复用开漏，100kHz标准模式，7位地址0x3C（库函数写地址传0x78）。轮询BUSY、START、地址、TBE和BTC标志；逐字节完成后检测NACK。每阶段最长5ms，检测总线错误、仲裁丢失及溢出；失败复位I2C外设并返回IO错误，不自动重试。硬件产生时钟和移位，CPU仍轮询等待，不是DMA或异步传输。DWT仅用于启动延时及超时计时，不再逐位控制GPIO；SystemCoreClock必须准确。外部器件持续拉低总线时，外设复位不能替代接线或器件故障处理。

- 引脚依据GD32F103xx数据手册PB10/PB11复用表，API及地址/标志清除行为依据本地Lib/Source/gd32f10x_i2c.c。官方数据手册：https://www.gd32mcu.com/data/documents/datasheet/GD32F103xx_Datasheet_Rev2.12.pdf 。

- 演示工程SysTick每1ms调用Key_Scan，主循环通过短临界区取事件，刷新屏幕时仍扫描按键。OLED操作只允许在主循环调用；不能并发访问显示缓冲和总线。

打开HMI.uvprojx选择HMI_Demo编译。原2.uvprojx的示例主程序保留。演示显示Demo Hz=1234.5678和整数示例，三个按键改变档位 / 模式 / 状态；这些只是人机接口自检，不是实际频率测量。

5. 验证与提交

Keil ARM Compiler 5.05 update 1完整编译链接：0 Error(s)，0 Warning(s)，生成Objects_HMI/HMI_Demo.hex。代码5224字节、只读数据828字节、RW24字节、ZI2064字节。详情见HMI_build.log。

主机测试：GCC C99，-Wall -Wextra -Werror；覆盖抖动、20ms阈值、长按单事件、同时按键、毫秒回绕、缓冲清屏 / 边界、INT32_MIN、浮点舍入、NaN / 大数 / 非法小数位、通信失败和初始化失败状态。新增硬件适配层模拟测试：I2C1选择、100kHz配置、0x78写地址、命令/数据/STOP顺序、地址/控制字节/末尾数据NACK、BUSY/START/地址/TBE/BTC/STOP超时、仲裁丢失、总线错误、DWT回绕，以及恢复后下一笔传输成功。模拟标志测试不替代真实I2C时序测量。

尚未验证：真实GPIO电平、按键波形、OLED供电 / 针序、I2C实际时序 / ACK、屏幕显示方向及刷新、SW4复位、烧录和实际测量链。编译及模拟总线测试不等于上板通过。

提交顺序：本分工说明PDF → key.h → key.c → oled.h → oled.c → hmi_port.h → hmi_port.c → hmi_demo.c（各文件独立PDF）→ 原始EDA导出PDF（3页）。总PDF与可编辑源码一并归档。

命名：项目班1-1直接测周法频率计基础模块02_人机接口何玉V1.1.pdf。班级名称取自本次原理图标题；若课程实际班级标识不同，提交前替换。

该合并文件为待原理图整改的材料稿，不能作为“全部考核项已完成”的声明。整改后重新导出图纸，再运行tools/build_hmi_pdf.py可按同一顺序重新合并；在成员实际完成评审和上板测试后更新验证记录。

修改历史：2026-10-09，V1.0为软件I2C；V1.1按要求切换为硬件I2C1，更新适配层测试、编译记录和提交材料。

key.h | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* 按键模块：提供三个独立按键初始化、非阻塞扫描、消抖和键值接口。 */
#ifndef HMI_KEY_H
#define HMI_KEY_H
#include <stdint.h>
typedef enum {
    KEY_NONE=0, KEY_TIMEBASE=1, KEY_MODE=2, KEY_HOLD=3
} key_value_t;
/**
 * 功能：初始化按键 GPIO 并复位消抖状态
 * 参数：无
 * 返回：无
 * 注意：启动时调用；按下状态从下一次扫描开始计时
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_Init。
 */
void Key_Init(void);
/**
 * 功能：非阻塞扫描三键，连续稳定20ms后更新状态
 * 参数：now_ms：单调递增毫秒节拍，允许32位回绕
 * 返回：无
 * 注意：每1~5ms调用；可在节拍中断调用；仅允许一个扫描调用者
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_Scan。
 */
void Key_Scan(uint32_t now_ms);
/**
 * 功能：取出一个消抖后的按下键值
 * 参数：无
 * 返回：KEY_NONE或三个功能键值
 * 注意：主循环调用；同一键未消费的多次事件合并；同时按下按键号顺序返回
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_GetValue。
 */
key_value_t Key_GetValue(void);
#endif
```

key.c | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* 按键模块：实现PA5/PA6/PA7低有效按键扫描、20ms消抖与按下事件。 */
#include "key.h"
#include "hmi_port.h"
#define KEY_COUNT 3U
#define DEBOUNCE_MS 20U
static uint8_t candidate[KEY_COUNT], stable[KEY_COUNT];
static volatile uint8_t pending;
static uint32_t changed_at[KEY_COUNT];
/**
 * 功能：初始化按键 GPIO 并复位消抖状态
 * 参数：无
 * 返回：无
 * 注意：启动时调用；按下状态从下一次扫描开始计时
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_Init。
 */
void Key_Init(void)
{
    uint8_t i;
    HMI_PortInit(); pending=0;
    for (i=0; i<KEY_COUNT; ++i) {
        candidate[i]=0; stable[i]=0; changed_at[i]=0;
    }
}
/**
 * 功能：非阻塞扫描三键，连续稳定20ms后更新状态
 * 参数：now_ms：单调递增毫秒节拍，允许32位回绕
 * 返回：无
 * 注意：每1~5ms调用；可在节拍中断调用；仅允许一个扫描调用者
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_Scan。
 */
void Key_Scan(uint32_t now_ms)
{
    uint8_t i, raw=HMI_KeyRead();
    for (i=0; i<KEY_COUNT; ++i) {
        uint8_t pressed=(uint8_t)((raw>>i)&1U);
        if (pressed!=candidate[i]) {
            candidate[i]=pressed; changed_at[i]=now_ms;
        } else if (stable[i]!=pressed &&
                   (uint32_t)(now_ms-changed_at[i])>=DEBOUNCE_MS) {
            stable[i]=pressed;
            if (pressed) pending|=(uint8_t)(1U<<i);
        }
    }
}
/**
 * 功能：取出一个消抖后的按下键值
 * 参数：无
 * 返回：KEY_NONE或三个功能键值
 * 注意：主循环调用；同一键未消费的多次事件合并；同时按下按键号顺序返回
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 Key_GetValue。
 */
key_value_t Key_GetValue(void)
{
    uint8_t i; uint32_t mask=HMI_EnterCritical();
    for (i=0; i<KEY_COUNT; ++i) {
        if (pending&(1U<<i)) {
            pending&=(uint8_t)~(1U<<i);
            HMI_ExitCritical(mask);
```

```
62              return (key_value_t)(i+1U);
63          }
64      }
65      HMI_ExitCritical(mask); return KEY_NONE;
66  }
```

oled.h | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* OLED模块：提供SSD1306初始化、清屏、字符/字符串、整数和浮点显示接口。 */
#ifndef HMI_OLED_H
#define HMI_OLED_H
#include <stdint.h>
typedef enum {
    OLED_OK=0, OLED_BAD_ARG=-1, OLED_IO_ERROR=-2, OLED_NOT_READY=-3
} oled_status_t;
/**
 * 功能：初始化SSD1306并清空屏幕
 * 参数：无
 * 返回：OLED_OK或OLED_IO_ERROR
 * 注意：调用前系统时钟已就绪；配置为128x64、0x3C、模块内置电荷泵
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_Init。
 */
oled_status_t OLED_Init(void);
/**
 * 功能：清空本地显示缓冲区
 * 参数：无
 * 返回：OLED_OK或OLED_NOT_READY
 * 注意：调用OLED_Refresh后实际屏幕才更新
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_Clear。
 */
oled_status_t OLED_Clear(void);
/**
 * 功能：将1024字节缓冲区按8页写入屏幕
 * 参数：无
 * 返回：OLED_OK/OLED_IO_ERROR/OLED_NOT_READY
 * 注意：阻塞通信，仅主循环调用；失败时可重试或重新初始化
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_Refresh。
 */
oled_status_t OLED_Refresh(void);
/**
 * 功能：在缓冲区显示一个5x7字符，含1列间距
 * 参数：x：0~122像素；page：0~7；ch：ASCII字符
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：只修改缓冲；不可显示字符替换为问号；再调用Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowChar。
 */
oled_status_t OLED_ShowChar(uint8_t x, uint8_t page, char ch);
/**
 * 功能：在缓冲区显示单行字符串
 * 参数：x：起始像素；page：页0~7；text：以零结束的字符串
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：超出本行返回错误且不写入；不自动换行；更新后Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowString。
 */
oled_status_t OLED_ShowString(uint8_t x, uint8_t page, const char *text);
/**
 * 功能：显示有符号32位十进制整数
 * 参数：x/page：坐标；value：整数，含INT32_MIN
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：不补前导零；覆盖旧数值前清屏或用空格填充；再Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowInt。
 */
oled_status_t OLED_ShowInt(uint8_t x, uint8_t page, int32_t value);
```

```
/**
 * 功能：显示定点格式浮点数，四舍五入
 * 参数：x/page：坐标；value：数值；decimals：0~6小数位
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：拒绝NaN/无穷和放大后超出uint32范围的值；舍入半值远离零；再Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowFloat。
 */
oled_status_t OLED_ShowFloat(uint8_t x, uint8_t page, double value, uint8_t decimals);
#endif
```

oled.c | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* OLED模块：实现SSD1306页缓冲、5x7 ASCII字库和数值格式化显示。 */
#include "oled.h"
#include "hmi_port.h"
#include <string.h>
static uint8_t framebuffer[8][128];
static uint8_t ready;
/* 原创5x7点阵；ASCII 32~126；每字符加1列间距。 */
static const uint8_t font5x7[95][5]={
    {0x00,0x00,0x00,0x00,0x00}, /* ASCII 32 */
    {0x00,0x00,0x5F,0x00,0x00}, /* ASCII 33 */
    {0x00,0x07,0x00,0x07,0x00}, /* ASCII 34 */
    {0x12,0x3F,0x12,0x3F,0x12}, /* ASCII 35 */
    {0x24,0x2A,0x7F,0x2A,0x12}, /* ASCII 36 */
    {0x13,0x0B,0x34,0x32,0x01}, /* ASCII 37 */
    {0x36,0x49,0x55,0x22,0x50}, /* ASCII 38 */
    {0x00,0x04,0x03,0x00,0x00}, /* ASCII 39 */
    {0x00,0x1C,0x22,0x41,0x00}, /* ASCII 40 */
    {0x00,0x41,0x22,0x1C,0x00}, /* ASCII 41 */
    {0x2A,0x1C,0x3E,0x1C,0x2A}, /* ASCII 42 */
    {0x08,0x08,0x3E,0x08,0x08}, /* ASCII 43 */
    {0x00,0x40,0x30,0x00,0x00}, /* ASCII 44 */
    {0x08,0x08,0x08,0x08,0x08}, /* ASCII 45 */
    {0x00,0x00,0x60,0x00,0x00}, /* ASCII 46 */
    {0x10,0x08,0x04,0x02,0x01}, /* ASCII 47 */
    {0x3E,0x51,0x49,0x45,0x3E}, /* ASCII 48 */
    {0x00,0x42,0x7F,0x40,0x00}, /* ASCII 49 */
    {0x42,0x61,0x51,0x49,0x46}, /* ASCII 50 */
    {0x41,0x49,0x49,0x49,0x36}, /* ASCII 51 */
    {0x18,0x14,0x12,0x7F,0x10}, /* ASCII 52 */
    {0x4F,0x49,0x49,0x49,0x31}, /* ASCII 53 */
    {0x3E,0x49,0x49,0x49,0x30}, /* ASCII 54 */
    {0x01,0x71,0x09,0x05,0x03}, /* ASCII 55 */
    {0x36,0x49,0x49,0x49,0x36}, /* ASCII 56 */
    {0x06,0x49,0x49,0x49,0x3E}, /* ASCII 57 */
    {0x00,0x00,0x36,0x00,0x00}, /* ASCII 58 */
    {0x00,0x40,0x36,0x00,0x00}, /* ASCII 59 */
    {0x08,0x14,0x22,0x41,0x00}, /* ASCII 60 */
    {0x14,0x14,0x14,0x14,0x14}, /* ASCII 61 */
    {0x00,0x41,0x22,0x14,0x08}, /* ASCII 62 */
    {0x02,0x01,0x51,0x09,0x06}, /* ASCII 63 */
    {0x3E,0x41,0x5D,0x55,0x1E}, /* ASCII 64 */
    {0x7E,0x09,0x09,0x09,0x7E}, /* ASCII 65 */
    {0x7F,0x49,0x49,0x49,0x36}, /* ASCII 66 */
    {0x3E,0x41,0x41,0x41,0x22}, /* ASCII 67 */
    {0x7F,0x41,0x41,0x41,0x3E}, /* ASCII 68 */
    {0x7F,0x49,0x49,0x49,0x41}, /* ASCII 69 */
    {0x7F,0x09,0x09,0x09,0x01}, /* ASCII 70 */
    {0x3E,0x41,0x49,0x49,0x3A}, /* ASCII 71 */
    {0x7F,0x08,0x08,0x08,0x7F}, /* ASCII 72 */
    {0x00,0x41,0x7F,0x41,0x00}, /* ASCII 73 */
    {0x20,0x40,0x41,0x3F,0x01}, /* ASCII 74 */
    {0x7F,0x08,0x14,0x22,0x41}, /* ASCII 75 */
    {0x7F,0x40,0x40,0x40,0x40}, /* ASCII 76 */
    {0x7F,0x02,0x0C,0x02,0x7F}, /* ASCII 77 */
    {0x7F,0x02,0x04,0x08,0x7F}, /* ASCII 78 */
    {0x3E,0x41,0x41,0x41,0x3E}, /* ASCII 79 */
    {0x7F,0x09,0x09,0x09,0x06}, /* ASCII 80 */
    {0x3E,0x41,0x51,0x21,0x5E}, /* ASCII 81 */
    {0x7F,0x09,0x19,0x29,0x46}, /* ASCII 82 */
    {0x46,0x49,0x49,0x49,0x31}, /* ASCII 83 */
    {0x01,0x01,0x7F,0x01,0x01}, /* ASCII 84 */
```

```
    {0x3F,0x40,0x40,0x40,0x3F}, /* ASCII 85 */
    {0x1F,0x20,0x40,0x20,0x1F}, /* ASCII 86 */
    {0x3F,0x40,0x38,0x40,0x3F}, /* ASCII 87 */
    {0x63,0x14,0x08,0x14,0x63}, /* ASCII 88 */
    {0x03,0x04,0x78,0x04,0x03}, /* ASCII 89 */
    {0x61,0x51,0x49,0x45,0x43}, /* ASCII 90 */
    {0x00,0x7F,0x41,0x41,0x00}, /* ASCII 91 */
    {0x01,0x02,0x04,0x08,0x10}, /* ASCII 92 */
    {0x00,0x41,0x41,0x7F,0x00}, /* ASCII 93 */
    {0x04,0x02,0x01,0x02,0x04}, /* ASCII 94 */
    {0x40,0x40,0x40,0x40,0x40}, /* ASCII 95 */
    {0x00,0x01,0x02,0x04,0x00}, /* ASCII 96 */
    {0x20,0x54,0x54,0x54,0x78}, /* ASCII 97 */
    {0x7F,0x44,0x44,0x44,0x38}, /* ASCII 98 */
    {0x38,0x44,0x44,0x44,0x28}, /* ASCII 99 */
    {0x38,0x44,0x44,0x44,0x7F}, /* ASCII 100 */
    {0x38,0x54,0x54,0x54,0x18}, /* ASCII 101 */
    {0x08,0x7E,0x09,0x01,0x02}, /* ASCII 102 */
    {0x0C,0x52,0x52,0x52,0x3E}, /* ASCII 103 */
    {0x7F,0x04,0x04,0x04,0x78}, /* ASCII 104 */
    {0x00,0x44,0x7D,0x40,0x00}, /* ASCII 105 */
    {0x20,0x40,0x44,0x3D,0x00}, /* ASCII 106 */
    {0x7F,0x10,0x28,0x44,0x00}, /* ASCII 107 */
    {0x00,0x41,0x7F,0x40,0x00}, /* ASCII 108 */
    {0x7C,0x04,0x78,0x04,0x78}, /* ASCII 109 */
    {0x7C,0x04,0x04,0x04,0x78}, /* ASCII 110 */
    {0x38,0x44,0x44,0x44,0x38}, /* ASCII 111 */
    {0x7E,0x12,0x12,0x12,0x0C}, /* ASCII 112 */
    {0x0C,0x12,0x12,0x12,0x7E}, /* ASCII 113 */
    {0x7C,0x08,0x04,0x04,0x08}, /* ASCII 114 */
    {0x48,0x54,0x54,0x54,0x24}, /* ASCII 115 */
    {0x04,0x3F,0x44,0x40,0x20}, /* ASCII 116 */
    {0x3C,0x40,0x40,0x20,0x7C}, /* ASCII 117 */
    {0x1C,0x20,0x40,0x20,0x1C}, /* ASCII 118 */
    {0x3C,0x40,0x30,0x40,0x3C}, /* ASCII 119 */
    {0x44,0x28,0x10,0x28,0x44}, /* ASCII 120 */
    {0x0E,0x50,0x50,0x50,0x3E}, /* ASCII 121 */
    {0x44,0x64,0x54,0x4C,0x44}, /* ASCII 122 */
    {0x00,0x08,0x36,0x41,0x00}, /* ASCII 123 */
    {0x00,0x00,0x7F,0x00,0x00}, /* ASCII 124 */
    {0x00,0x41,0x36,0x08,0x00}, /* ASCII 125 */
    {0x08,0x04,0x08,0x08,0x04}, /* ASCII 126 */
};
/**
 * 功能：将uint32转换为十进制字符
 * 参数：value：无符号值；text：至少11字节缓冲
 * 返回：字符数，不含结束零
 * 注意：内部使用，不调用printf
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 format_uint。
 */
static uint8_t format_uint(uint32_t value, char *text)
{
    char reverse[10]; uint8_t n=0,i;
    do { reverse[n++]=(char)('0'+value%10U); value/=10U; } while (value);
    for (i=0;i<n;++i) text[i]=reverse[n-1U-i];
    text[n]='\0'; return n;
}
/**
 * 功能：初始化SSD1306并清空屏幕
 * 参数：无
```

```
 * 返回：OLED_OK或OLED_IO_ERROR
 * 注意：调用前系统时钟已就绪；配置为128x64、0x3C、模块内置电荷泵
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_Init。
 */
oled_status_t OLED_Init(void)
{
    static const uint8_t config[]={
        0xAE,0xD5,0x80,0xA8,0x3F,0xD3,0x00,0x40,
        0x8D,0x14,0x20,0x02,0xA1,0xC8,0xDA,0x12,
        0x81,0x7F,0xD9,0xF1,0xDB,0x40,0xA4,0xA6,0x2E
    };
    const uint8_t on=0xAF;
    ready=0; HMI_PortInit(); HMI_DelayMs(100);
    if (HMI_OLEDWrite(0x00,config,sizeof(config))!=0)
        return OLED_IO_ERROR;
    ready=1; (void)OLED_Clear();
    if (OLED_Refresh()!=OLED_OK || HMI_OLEDWrite(0x00,&on,1)!=0) {
        ready=0; return OLED_IO_ERROR;
    }
    return OLED_OK;
}
/**
 * 功能：清空本地显示缓冲区
 * 参数：无
 * 返回：OLED_OK或OLED_NOT_READY
 * 注意：调用OLED_Refresh后实际屏幕才更新
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_Clear。
 */
oled_status_t OLED_Clear(void)
{
    if (!ready) return OLED_NOT_READY;
    memset(framebuffer,0,sizeof(framebuffer)); return OLED_OK;
}
/**
 * 功能：将1024字节缓冲区按8页写入屏幕
 * 参数：无
 * 返回：OLED_OK/OLED_IO_ERROR/OLED_NOT_READY
 * 注意：阻塞通信，仅主循环调用；失败时可重试或重新初始化
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_Refresh。
 */
oled_status_t OLED_Refresh(void)
{
    uint8_t p;
    if (!ready) return OLED_NOT_READY;
    for (p=0;p<8;++p) {
        uint8_t position[3]={(uint8_t)(0xB0U+p),0x00,0x10};
        if (HMI_OLEDWrite(0x00,position,3)!=0 ||
            HMI_OLEDWrite(0x40,framebuffer[p],128)!=0)
            return OLED_IO_ERROR;
    }
    return OLED_OK;
}
/**
 * 功能：在缓冲区显示一个5x7字符，含1列间距
 * 参数：x：0~122像素；page：0~7；ch：ASCII字符
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：只修改缓冲；不可显示字符替换为问号；再调用Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
```

```
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowChar。
 */
oled_status_t OLED_ShowChar(uint8_t x, uint8_t page, char ch)
{
    uint8_t code=(uint8_t)ch;
    if (!ready) return OLED_NOT_READY;
    if (x>122 || page>7) return OLED_BAD_ARG;
    if (code<32 || code>126) code='?';
    memcpy(&framebuffer[page][x],font5x7[code-32U],5);
    framebuffer[page][x+5U]=0; return OLED_OK;
}
/**
 * 功能：在缓冲区显示单行字符串
 * 参数：x：起始像素；page：页0~7；text：以零结束的字符串
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：超出本行返回错误且不写入；不自动换行；更新后Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowString。
 */
oled_status_t OLED_ShowString(uint8_t x, uint8_t page, const char *text)
{
    uint8_t n=0,i,capacity;
    if (!ready) return OLED_NOT_READY;
    if (text==0 || page>7 || x>122) return OLED_BAD_ARG;
    capacity=(uint8_t)((128U-x)/6U);
    while (text[n]!=0) {
        if (n>=capacity) return OLED_BAD_ARG;
        ++n;
    }
    for (i=0;i<n;++i) (void)OLED_ShowChar((uint8_t)(x+i*6U),page,text[i]);
    return OLED_OK;
}
/**
 * 功能：显示有符号32位十进制整数
 * 参数：x/page：坐标；value：整数，含INT32_MIN
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：不补前导零；覆盖旧数值前清屏或用空格填充；再Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowInt。
 */
oled_status_t OLED_ShowInt(uint8_t x, uint8_t page, int32_t value)
{
    char text[12]; uint8_t offset=0; uint32_t magnitude;
    if (value<0) { text[offset++]='-'; magnitude=(uint32_t)(-(value+1))+1U; }
    else magnitude=(uint32_t)value;
    (void)format_uint(magnitude,text+offset);
    return OLED_ShowString(x,page,text);
}
/**
 * 功能：显示定点格式浮点数，四舍五入
 * 参数：x/page：坐标；value：数值；decimals：0~6小数位
 * 返回：OLED_OK/OLED_BAD_ARG/OLED_NOT_READY
 * 注意：拒绝NaN/无穷和放大后超出uint32范围的值；舍入半值远离零；再Refresh
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 OLED_ShowFloat。
 */
oled_status_t OLED_ShowFloat(uint8_t x, uint8_t page, double value, uint8_t decimals)
{
    char text[24]; uint32_t scale=1,rounded,fraction;
    uint8_t i,n=0; double magnitude=value;
    if (decimals>6 || value!=value) return OLED_BAD_ARG;
```

```
    if (value<0) magnitude=-value;
    for (i=0;i<decimals;++i) scale*=10U;
    magnitude*=scale;
    if (!(magnitude<=4294967294.0)) return OLED_BAD_ARG;
    rounded=(uint32_t)(magnitude+0.5);
    if (value<0 && rounded!=0) text[n++]='-';
    n+=format_uint(rounded/scale,text+n);
    if (decimals) {
        text[n++]='.'; fraction=rounded%scale;
        for (i=0;i<decimals;++i) {
            scale/=10U; text[n++]=(char)('0'+fraction/scale); fraction%=scale;
        }
    }
    text[n]=0; return OLED_ShowString(x,page,text);
}
```

hmi_port.h | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* 硬件适配模块：封装PA5~PA7按键读取、PB10/PB11硬件I2C1和短延时。 */
#ifndef HMI_PORT_H
#define HMI_PORT_H
#include <stdint.h>
#define HMI_OLED_ADDRESS 0x3CU /* 7位地址 */
/**
 * 功能：初始化按键输入与OLED硬件I2C1（100kHz）及DWT计时
 * 参数：无
 * 返回：无
 * 注意：幂等；先完成系统时钟初始化；保留PA0/PA1及SWD
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_PortInit；V1.1，改用硬件I2C1。
 */
void HMI_PortInit(void);
/**
 * 功能：读取三个功能键的瞬时按下状态
 * 参数：无
 * 返回：bit0~2对应SW1~SW3；1表示按下
 * 注意：内部上拉，未消抖；SW4为硬件复位不参与扫描
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_KeyRead。
 */
uint8_t HMI_KeyRead(void);
/**
 * 功能：按毫秒执行短时阻塞延时
 * 参数：ms：延时毫秒数
 * 返回：无
 * 注意：仅OLED启动使用；需先初始化DWT；不关闭中断
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_DelayMs。
 */
void HMI_DelayMs(uint32_t ms);
/**
 * 功能：硬件I2C1写SSD1306，检查NACK、总线错误和5ms阶段超时
 * 参数：control：0x00命令/0x40数据；data：数据；len：1~128字节
 * 返回：0成功；-1总线超时/NACK/非法参数
 * 注意：仅主循环；PB10/PB11开漏，需外部3.3V上拉；失败复位外设，不重试
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_OLEDWrite；V1.1，改用硬件I2C1。
 */
int HMI_OLEDWrite(uint8_t control, const uint8_t *data, uint16_t len);
/**
 * 功能：保存中断屏蔽状态并进入短临界区
 * 参数：无
 * 返回：原PRIMASK值
 * 注意：仅保护取键值，不在临界区进行OLED传输
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_EnterCritical。
 */
uint32_t HMI_EnterCritical(void);
/**
 * 功能：恢复进入临界区前的中断状态
 * 参数：mask：EnterCritical返回值
 * 返回：无
 * 注意：必须与EnterCritical成对使用
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_ExitCritical。
 */
void HMI_ExitCritical(uint32_t mask);
#endif
```

hmi_port.c | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* 硬件适配模块：实现GD32 GPIO、硬件I2C1、ACK/超时检查和DWT延时。 */
#include "hmi_port.h"
#include "gd32f10x.h"
#define SCL GPIO_PIN_10
#define SDA GPIO_PIN_11
static uint8_t initialized;
/**
 * 功能：用DWT周期计数提供微秒延时
 * 参数：us：微秒数（内部仅1000）
 * 返回：无
 * 注意：SystemCoreClock必须准确，DWT需保持运行
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 delay_us。
 */
static void delay_us(uint32_t us)
{
    uint32_t start=DWT->CYCCNT;
    uint32_t cycles=(SystemCoreClock/1000000U)*us;
    while ((uint32_t)(DWT->CYCCNT-start)<cycles) {}
}
/**
 * 功能：配置或复位I2C1为100kHz主机。
 * 参数：无。返回：无。
 * 注意：复位清除外设状态，无法解除外部器件持续拉低的总线。
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_configure。
 */
static void i2c_configure(void)
{
    i2c_deinit(I2C1);
    i2c_clock_config(I2C1,100000U,I2C_DTCY_2);
    /* 本机地址仅用于外设配置，不是OLED从机地址。 */
    i2c_mode_addr_config(I2C1,I2C_I2CMODE_ENABLE,I2C_ADDFORMAT_7BITS,0x00U);
    i2c_enable(I2C1);
    i2c_ack_config(I2C1,I2C_ACK_ENABLE);
}
/**
 * 功能：等待硬件I2C标志，同时检查NACK、总线错误和仲裁丢失。
 * 参数：flag：目标标志；expected：SET或RESET。
 * 返回：0成功；-1错误或超时。
 * 注意：每阶段最长5ms，DWT需运行；不屏蔽中断，计数差支持回绕。
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_wait。
 */
static int i2c_wait(i2c_flag_enum flag, FlagStatus expected)
{
    uint32_t start=DWT->CYCCNT;
    uint32_t cycles=(SystemCoreClock/1000000U)*5000U;
    for (;;) {
        if (SET==i2c_flag_get(I2C1,I2C_FLAG_AERR) ||
            SET==i2c_flag_get(I2C1,I2C_FLAG_BERR) ||
            SET==i2c_flag_get(I2C1,I2C_FLAG_LOSTARB) ||
            SET==i2c_flag_get(I2C1,I2C_FLAG_OUERR)) return -1;
        if (expected==i2c_flag_get(I2C1,flag)) return 0;
        if ((uint32_t)(DWT->CYCCNT-start)>=cycles) return -1;
    }
}
/**
 * 功能：异常退出传输并复位I2C1状态。
 * 参数：无。返回：无。
 * 注意：只有仍为主机且未丢失仲裁时才发STOP；不重试原数据。
```

```
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_abort。
 */
static void i2c_abort(void)
{
    if (RESET==i2c_flag_get(I2C1,I2C_FLAG_LOSTARB) &&
        SET==i2c_flag_get(I2C1,I2C_FLAG_MASTER)) i2c_stop_on_bus(I2C1);
    i2c_configure();
}
/**
 * 功能：发送一个字节并等待完整传输及ACK结果。
 * 参数：value：待发字节。返回：0成功；-1错误或超时。
 * 注意：BTC表示移位器及数据寄存器完成；仅TBE不能证明收到ACK。
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_send_byte。
 */
static int i2c_send_byte(uint8_t value)
{
    if (i2c_wait(I2C_FLAG_TBE,SET)!=0) return -1;
    i2c_data_transmit(I2C1,value);
    return i2c_wait(I2C_FLAG_BTC,SET);
}
/**
 * 功能：初始化按键输入、100kHz硬件I2C1及DWT计时
 * 参数：无
 * 返回：无
 * 注意：幂等；先完成系统时钟初始化；保留PA0/PA1及SWD
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_PortInit；V1.1，改用硬件I2C1。
 */
void HMI_PortInit(void)
{
    if (initialized) return;
    rcu_periph_clock_enable(RCU_GPIOA);
    rcu_periph_clock_enable(RCU_GPIOB);
    rcu_periph_clock_enable(RCU_AF);
    rcu_periph_clock_enable(RCU_I2C1);
    gpio_init(GPIOA,GPIO_MODE_IPU,GPIO_OSPEED_2MHZ,
              GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7);
    gpio_init(GPIOB,GPIO_MODE_AF_OD,GPIO_OSPEED_50MHZ,SCL|SDA);
    CoreDebug->DEMCR|=CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CTRL|=DWT_CTRL_CYCCNTENA_Msk;
    i2c_configure(); initialized=1;
}
/**
 * 功能：读取三个功能键的瞬时按下状态
 * 参数：无
 * 返回：bit0~2对应SW1~SW3；1表示按下
 * 注意：内部上拉，未消抖；SW4为硬件复位不参与扫描
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_KeyRead。
 */
uint8_t HMI_KeyRead(void)
{
    uint8_t bits=0;
    if (RESET==gpio_input_bit_get(GPIOA,GPIO_PIN_5)) bits|=1;
    if (RESET==gpio_input_bit_get(GPIOA,GPIO_PIN_6)) bits|=2;
    if (RESET==gpio_input_bit_get(GPIOA,GPIO_PIN_7)) bits|=4;
    return bits;
}
/**
```

```
 * 功能：按毫秒执行短时阻塞延时
 * 参数：ms：延时毫秒数
 * 返回：无
 * 注意：仅OLED启动使用；需先初始化DWT；不关闭中断
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_DelayMs。
 */
void HMI_DelayMs(uint32_t ms)
{
    while (ms--) delay_us(1000);
}
/**
 * 功能：硬件I2C1轮询写SSD1306，检查NACK、总线错误和5ms阶段超时
 * 参数：control：0x00命令/0x40数据；data：数据；len：1~128字节
 * 返回：0成功；-1总线超时/NACK/非法参数
 * 注意：仅主循环；PB10/PB11复用开漏，需外部3.3V上拉；失败复位外设，不重试
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_OLEDWrite；V1.1，改用硬件I2C1。
 */
int HMI_OLEDWrite(uint8_t control, const uint8_t *data, uint16_t len)
{
    uint16_t i;
    if (!initialized || data==0 || len==0 || len>128 ||
        (control!=0x00 && control!=0x40)) return -1;
    if (i2c_wait(I2C_FLAG_I2CBSY,RESET)!=0) goto fail;
    i2c_start_on_bus(I2C1);
    if (i2c_wait(I2C_FLAG_SBSEND,SET)!=0) goto fail;
    /* 库函数要求左移一位的地址；0x3C对应写地址0x78。 */
    i2c_master_addressing(I2C1,(uint32_t)(HMI_OLED_ADDRESS<<1),I2C_TRANSMITTER);
    if (i2c_wait(I2C_FLAG_ADDSEND,SET)!=0) goto fail;
    i2c_flag_clear(I2C1,I2C_FLAG_ADDSEND);
    if (i2c_send_byte(control)!=0) goto fail;
    for (i=0;i<len;++i) {
        if (i2c_send_byte(data[i])!=0) goto fail;
    }
    i2c_stop_on_bus(I2C1);
    if (i2c_wait(I2C_FLAG_I2CBSY,RESET)!=0) goto fail;
    return 0;
fail:
    i2c_abort(); return -1;
}
/**
 * 功能：保存中断屏蔽状态并进入短临界区
 * 参数：无
 * 返回：原PRIMASK值
 * 注意：仅保护取键值，不在临界区进行OLED传输
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_EnterCritical。
 */
uint32_t HMI_EnterCritical(void)
{
    uint32_t mask=__get_PRIMASK(); __disable_irq(); return mask;
}
/**
 * 功能：恢复进入临界区前的中断状态
 * 参数：mask：EnterCritical返回值
 * 返回：无
 * 注意：必须与EnterCritical成对使用
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_ExitCritical。
 */
```

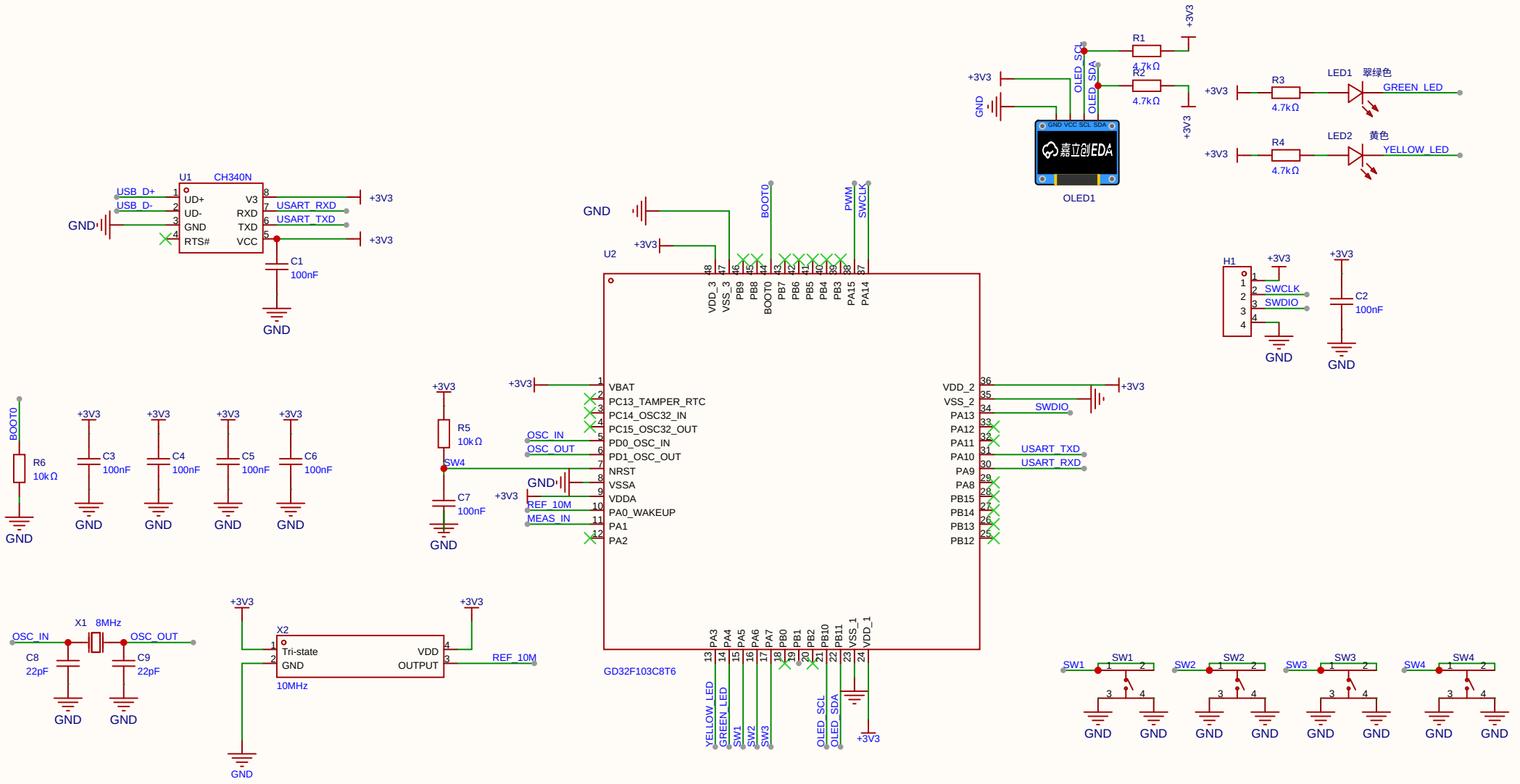
```
184 void HMI_ExitCritical(uint32_t mask)
185 {
186     __set_PRIMASK(mask);
187 }
```

hmi_demo.c | 基础模块02_人机接口

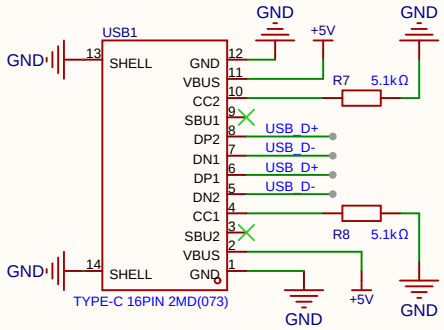
源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

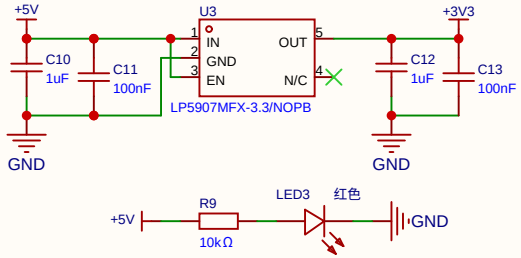
```c
/* 人机接口演示模块：提供1ms按键扫描、OLED基本显示与三个功能键交互。 */
#include "gd32f10x.h"
#include "key.h"
#include "oled.h"
static volatile uint32_t ticks;
/* 可在调试器观察；显示故障不阻止按键继续扫描。 */
volatile oled_status_t hmi_display_status;
/**
 * 功能：更新毫秒节拍并扫描按键
 * 参数：无
 * 返回：无
 * 注意：仅此中断调用Key_Scan，主循环只取事件
 * 作者/责任成员：蒋观海；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 SysTick_Handler。
 */
void SysTick_Handler(void)
{
    ++ticks; Key_Scan(ticks);
}
/**
 * 功能：绘制驱动演示页面
 * 参数：timebase：档0~2；mode：模式0~1；hold：保持状态
 * 返回：OLED状态码
 * 注意：演示数值不代表测量结果；统一刷新；坐标固定有效
 * 作者/责任成员：何玉；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 render_page。
 */
static oled_status_t render_page(uint8_t timebase, uint8_t mode, uint8_t hold)
{
    static const char *const bases[]={"10 MHz","1 MHz","100 kHz"};
    oled_status_t status=OLED_Clear();
    if (status!=OLED_OK) return status;
    (void)OLED_ShowString(0,0,"HMI DRIVER DEMO");
    (void)OLED_ShowString(0,1,"Demo Hz: ");
    (void)OLED_ShowFloat(54,1,1234.5678,4);
    (void)OLED_ShowString(0,2,"Base: ");
    (void)OLED_ShowString(36,2,bases[timebase]);
    (void)OLED_ShowString(0,3,mode?"Mode: MULTI":"Mode: SINGLE");
    (void)OLED_ShowString(0,4,hold?"State: HOLD":"State: RUN");
    (void)OLED_ShowString(0,5,"Int: ");
    (void)OLED_ShowInt(30,5,-2147483647-1);
    (void)OLED_ShowChar(0,6,'A');
    (void)OLED_ShowString(0,7,"SW1 Base 2 Mode 3 Run");
    return OLED_Refresh();
}
/**
 * 功能：启动人机接口演示并响应三个按键
 * 参数：无
 * 返回：嵌入式主程序不返回
 * 注意：按键先初始化再开启节拍；OLED失败每1秒重试；非测频程序
 * 作者/责任成员：蒋观海；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 main。
 */
int main(void)
{
    uint8_t base=0,mode=0,hold=0,dirty=1;
    uint32_t retry_at=0; key_value_t key;
    Key_Init();
    if (SysTick_Config(SystemCoreClock/1000U)!=0) while (1) {}
    hmi_display_status=OLED_Init();
    while (1) {
```

```
        while ((key=Key_GetValue())!=KEY_NONE) {
            if (key==KEY_TIMEBASE) base=(uint8_t)((base+1U)%3U);
            else if (key==KEY_MODE) mode^=1U;
            else if (key==KEY_HOLD) hold^=1U;
            dirty=1;
        }
        if (hmi_display_status!=OLED_OK &&
            (uint32_t)(ticks-retry_at)>=1000U) {
            retry_at=ticks; hmi_display_status=OLED_Init(); dirty=1;
        }
        if (dirty && hmi_display_status==OLED_OK) {
            hmi_display_status=render_page(base,mode,hold); dirty=0;
            retry_at=ticks;
        }
    }
}
```

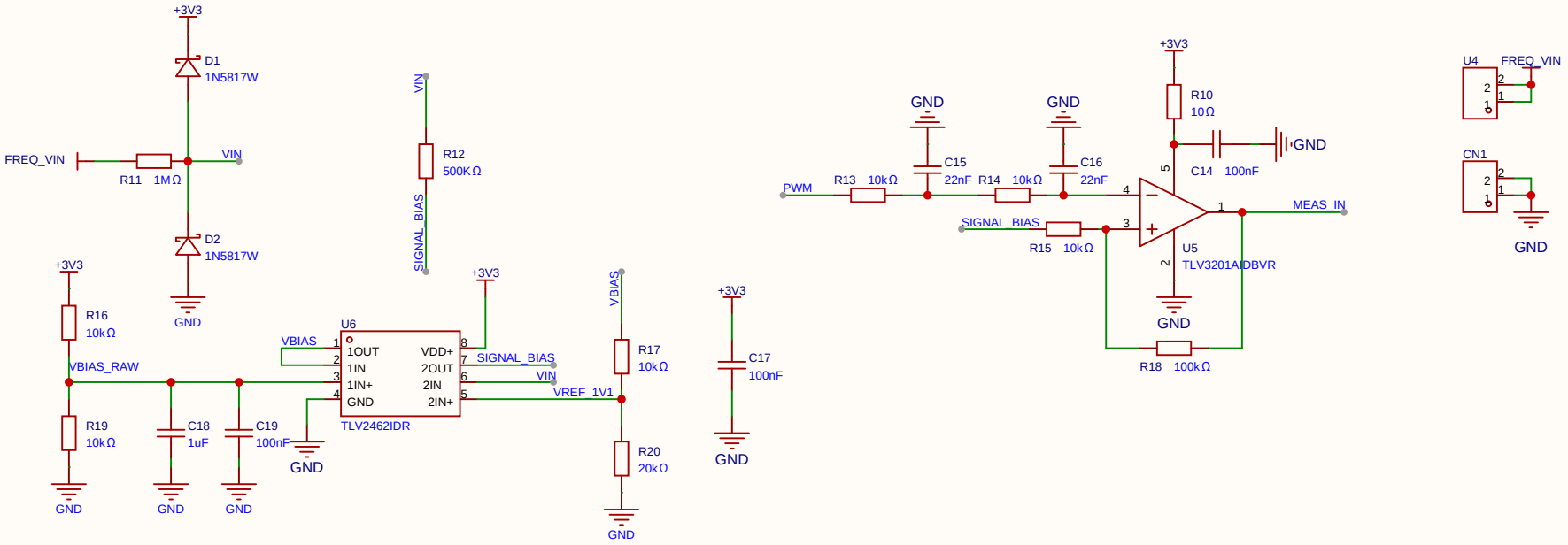


原理图	Schematic1_1	创建日期	2026-09-21
		更新日期	2026-10-09
板子	Board1_1	图页	主控
绘制		团队工程_项目班1-1_直接测周法频率计_蒋观海	
审阅			
		版本	尺寸
		V1.0	A4
嘉立创EDA		页 1 共 3	
		嘉立创EDA	





原理图	Schematic1_1	创建日期	2026-09-21
		更新日期	2026-09-29
板子	Board1_1	图页	type-c供电
绘制		团队工程_项目班1-1_直接测周法频率计_蒋观海	
审阅			
		版本	尺寸	页 2 共 3
嘉立创EDA		V1.0	A4	嘉立创EDA



原理图	Schematic1_1	创建日期	2026-09-21
		更新日期	2026-09-29
板子	Board1_1	图页	放大处理电路
绘制		团队工程_项目班1-1_直接测周法频率计_蒋观海	
审阅			
		版本	尺寸	页 3 共 3
嘉立创EDA		V1.0	A4	嘉立创EDA